CON10X Web Domain

Complete Engineering Reference

Visual Studio 2022 Solution, Client, Server, and Chrome Extension

A full code breakdown for a developer joining the project: how to open and build it, how the three parts fit together, and how every pillar, service and file works.
The first proof of ConteX Law applied to deterministic code generation.

Russel Hawkins | SnapStak.ai

Contents

1. Introduction

1.1 What this software does

CON10X Web Domain captures a live web page exactly as the browser has rendered it and regenerates it as production-ready source code in the framework you choose - React, Next.js, Vue, Nuxt, Svelte, Angular, SolidJS, Astro, Alpine, or vanilla JavaScript. There is no design tool in the loop and no guesswork: the same page captured under the same conditions produces the same code every time. That reproducibility is the whole point, and it is engineered deliberately rather than hoped for.

1.2 Why it is built this way: ConteX Law

The engine is the first working proof of ConteX Law - the principle that a generative model stops fabricating when its input is specified completely. Rather than asking a model to "build a page that looks like this," CON10X measures the page into four complete, non-overlapping descriptions (the four pillars) and gives the model a specification with no gaps left to fill. The model transcribes instead of inventing. Almost every design decision in this codebase follows from that one idea, so hold it in mind throughout.

Pillar	Question it answers	What it contains
Structure	What the thing IS	Exact geometry, hierarchy, colours and typography, serialised as an SVG treated as immutable truth.
Behaviour	What it DOES	Its CSS and JavaScript, turned into semantic descriptions by an AI pass.
Influence	What SHAPED it	Browser, OS, viewport, device pixel ratio, and the colour-scheme and reduced-motion preferences at capture time.
Objective	What it must BECOME	The target framework, device, screen width and real breakpoints the developer selects.

The consequence: the AI is called only once for the actual code, and only at the very end, with a fully specified prompt. Everything before that step exists to make that prompt complete. That is ConteX Law expressed as a pipeline.

1.3 Who this document is for

This is written for a developer who has just been handed the repository and needs to become productive. It assumes you can read C#, JavaScript and a little Blazor, but it assumes nothing about this project. It begins with how to open and build the solution in Visual Studio 2022, then explains the architecture, then walks every part in detail, and closes with reference appendices for the storage API and the data model.

2. The Visual Studio 2022 Solution

The repository is a Visual Studio 2022 solution targeting **.NET 9**. Open `SnapStak.Wasm.sln` in Visual Studio 2022 (version 17.x) and the whole system loads as two projects. The Chrome extension is a separate, plain folder of JavaScript that is not part of the .NET solution - it is loaded into Chrome directly. Understanding this split is the first thing a new developer needs.

2.1 Prerequisites

- **Visual Studio 2022** (17.8 or later) with the "ASP.NET and web development" workload installed. The solution format header declares Visual Studio Version 17.
- **.NET 9 SDK** - both projects set `<TargetFramework>net9.0</TargetFramework>`.
- **The Blazor WebAssembly build tools** (installed with the ASP.NET workload). The client uses the `Microsoft.NET.Sdk.BlazorWebAssembly` SDK.
- **Google Chrome** (or any Chromium browser) for loading the extension and running the capture engine, which depends on the Chrome DevTools Protocol.
- **Node.js** - only needed if you intend to encrypt the capture engine for a production publish (section 9.3). Not needed for normal development.

2.2 Solution structure

The solution contains two projects. The third part of the system, the extension, sits beside them in the repository but is not referenced by the build.

Item	Type	Role
SnapStak.Wasm.Server	ASP.NET Core (Microsoft.NET.Sdk.Web)	The startup project. Hosts the WASM client, relays snapshots, runs the disk pipeline and integrations. Listens on :5174.
SnapStak.Wasm.Client	Blazor WebAssembly (Microsoft.NET.Sdk.BlazorWebAssembly)	The application that runs in the browser. Owns the four pillars, the AI calls and the code generation.
SnapStak.Extension	Plain JS folder (not in the .sln)	Manifest V3 Chrome extension. Loaded into Chrome separately as an unpacked extension.
Figma snapstak-importer	Plain TS folder (not in the .sln)	A standalone Figma plugin that consumes the master SVG. Independent of the .NET build.

2.3 The project reference and build order

The Server has a project reference to the Client, and this is one of the most important build facts in the solution. It is declared in `SnapStak.Wasm.Server.csproj` as:

```
<ProjectReference
  Include="..\SnapStak.Wasm.Client\SnapStak.Wasm.Client.csproj"
  ReferenceOutputAssembly="true" />
```

With `ReferenceOutputAssembly="true"`, the Server consumes the Client's compiled assembly as a normal C# library. This is why the server can run the same pillar models and translator plugins as the browser: types like `IConteXTranslatorPlugin`, `TranslatorPluginHost` and the SVG node model live in the Client project and are linked into the Server at build time. The Blazor code that only the browser executes still ships through `wwwroot`; this flag only affects what is linked as a library.

Because of that reference, the Client must finish building before the Server compiles. The Server project forces this with `<BuildProjectReferences>true</BuildProjectReferences>`, which exists to fix an intermittent MSBuild race (errors CS0006 / RAZORSDK1007) where the Server's C# compiler could start before the Client's reference assembly existed in `obj/Debug/net9.0/ref/`. If you ever see a missing-reference error on a clean build, this setting is the relevant context. Build the Client first; the solution is configured to do so automatically.

2.4 NuGet packages

Add or update packages only through the Visual Studio 2022 NuGet Package Manager UI (right-click the project, Manage NuGet Packages, Browse, search, Install). Do not hand-edit the .csproj files and do not use the dotnet CLI for packages. The current package set is small and deliberate.

Client packages

Package	Version	Why it is here
Microsoft.AspNetCore.Components.WebAssembly	9.0.0	The Blazor WebAssembly runtime.
Microsoft.AspNetCore.Components.WebAssembly.DevServer	9.0.0	Local dev server for the WASM app (PrivateAssets=all - dev only).
Newtonsoft.Json	13.0.3	All JSON serialisation across the pillars and AI responses.
SauceControl.Blake2Fast	2.0.0	Fast hashing used in the storage / integrity paths.
SharpZipLib	1.4.2	Building the output project zip in memory.

Server packages

Package	Version	Why it is here
Microsoft.AspNetCore.Components.WebAssembly.Server	9.0.0	Serves the Blazor WASM client and its framework files.
Newtonsoft.Json	13.0.3	JSON serialisation for the snapshot relay and pipeline.
SkiaSharp	3.119.2	Server-side rasterisation in the disk pipeline.

2.5 Build configurations and what they change

The solution defines Debug and Release for Any CPU. The configuration does more than usual here because it flips two security behaviours, decided in code by the host environment:

- **Pillar encryption.** In Development `PillarEncryption` is constructed with `enabled:false` (data stored readable, prefixed with "plain:"); in production with `enabled:true` (real AES-GCM). This single flag makes local debugging possible without decrypting everything.
- **Engine encryption.** A Release publish ships the capture engine as an encrypted `content.enc` instead of plaintext `content.js`. See section 9.3.

2.6 Running the full system locally

Three things must run together. The order matters.

1. **Set the startup project to SnapStak.Wasm.Server and press F5 (or Ctrl+F5).** The server builds the Client first, then starts on http://localhost:5174 and serves the WASM app. In Development, encryption is off so pillar data is readable in the browser's IndexedDB for debugging.
2. **Load the extension.** In Chrome, open chrome://extensions, enable Developer mode, click "Load unpacked", and select the SnapStak.Extension folder. The extension expects the server at localhost:5174 and reports offline until the server is running.
3. **Complete Get Started in the app.** Open the WASM app, enter your subscription code and OpenRouter API key. The key is stored only in your browser and is used to call the AI directly.
4. **Capture and convert.** Open any web page, click the SnapStak extension icon, click Deconstruct. The snapshot streams into the app. Pick a framework and click Convert to generate and download the code.

3. Architecture Overview

3.1 The three parts

Three cooperating processes make up the system. The same concepts (pillars, components, SVG) appear in all three, so the first skill is always knowing which process you are reasoning about.

Part	Project / folder	Runs where	Responsible for
Chrome extension	`SnapStak.Extension`	In the browser, on the target page	Capturing the live DOM: Structure, Behaviour source, Influence, and the inferred Objective.
Client (WASM)	`SnapStak.Wasm.Client`	Browser, as a Blazor WebAssembly app	Orchestrating the four pillars, calling the AI, building the code zip. The user-facing app.
Server	`SnapStak.Wasm.Server`	Local ASP.NET Core host on :5174	Hosting the WASM app, relaying snapshots over SSE, running the filesystem pipeline and the design-tool / mobile relays.

Two pipelines, one logic. The same pillar logic exists twice. The client (Blazor WASM) runs the canonical pipeline the user drives, storing pillar files in the browser's IndexedDB. The server runs a parallel pipeline (`SnapStakPipeline.cs`) that writes the same pillar files to disk and drives the design-tool plugins. They share models and the plugin contract through the project reference. When you see the same concept twice, this is why - and your first question on any bug should be which pipeline you are in.

3.2 The end-to-end flow

Follow one component all the way through once. Every file you read later slots into one of these steps.

5. **Trigger.** The user clicks Deconstruct (or selects one component) in the extension popup.
6. **Inject.** The extension's background service worker fetches the capture engine (content.js) from the local server and injects it into the page's MAIN world through bridge.js.
7. **Capture.** content.js walks the rendered DOM with getBoundingClientRect() and getComputedStyle(), producing Structure geometry, Behaviour source (CSS/JS), the Influence environment, and a Phase-1 Objective. A second pass under mobile emulation captures the responsive viewport.
8. **Forward.** background.js assembles one JSON payload, stamps a componentId of the form hostname_unixtimestamp, and POSTs it to /api/snapshot.
9. **Relay.** The server relays the snapshot to the WASM client over Server-Sent Events, and in parallel runs its disk transform on a background thread.
10. **Build pillars.** The client's StructureAgentService turns the DOM into the master SVG (Structure) and writes the Behaviour source, Influence and Objective into IndexedDB, encrypted in production.
11. **Convert.** On Convert, the ConstructorService loads the four pillars, runs Behaviour AI to describe the CSS/JS, assembles the single ConteX Law prompt, and calls the model through the user's own OpenRouter key.

12. **Deliver.** The model returns componentCode and componentCSS as JSON. The Constructor parses it, names the component, builds a project zip in memory, and the browser downloads it.

3.3 Where the data lives

Store	Location	Holds
IndexedDB (browser)	Client, via `SnapStakIdb.js`	All pillar files for the canonical pipeline, encrypted. Keyed per user, per component.
localStorage (browser)	Client, via `LicenceService`	API key, user UUID, subscription id/token/status, encryption salt.
Filesystem	Server output root (default `C:\ConteX Dev\SnapStak.Wasm\output`)	The server pipeline's mirror of every component's pillar files, plus plugin outputs.
snapstak_settings.json	Beside the server executable	Output path and plugin toggles, synced from the Settings page.

4. The Chrome Extension (Capture Layer)

Location: `SnapStak.Extension/`. A Manifest V3 extension whose only job is to extract the four pillars' raw material from a live page. It generates no code and persists nothing - it captures and forwards.

Files: `manifest.json`, `background.js` (service worker), `bridge.js` (content script), `popup.html / popup.js` (the toolbar UI), `rules/csp_rules.json` (header rules), and the icons. The capture engine itself, `content.js`, is served from the client's wwwroot and injected at runtime - it is not bundled in the extension.

4.1 manifest.json - capabilities and why they exist

The manifest declares a deliberately powerful permission set. Each permission maps to a concrete capture need.

Permission	Why it is needed
debugger	Attaches the Chrome DevTools Protocol to emulate a phone (Emulation.setDeviceMetricsOverride) so the responsive viewport is captured from the same page.
scripting	Injects the bridge that loads the capture engine into the page's MAIN world.
declarativeNetRequest (+ feedback, withHostAccess)	Strips Content-Security-Policy response headers (rules/csp_rules.json) so the engine script can be injected on CSP-strict pages.
activeTab + host_permissions (http/https, localhost:5174)	Access to the page under capture and to the local server.
storage	Persists per-tab "engine ready" state in session storage so it survives the service worker sleeping.

Flag for review: the CSP-stripping rule and the debugger permission are the two most sensitive parts of the extension. They make capture work on real-world sites, but they are also why the extension requests strong permissions. Keep them front of mind for any security review or store submission.

4.2 The injection chain: three execution contexts

Capture spans three contexts that cannot call each other directly. The flow is a relay; getting this model right removes most of the confusion in the extension.

Script	Context	Role
background.js	Service worker	Orchestrates: receives popup clicks, injects the engine, runs the mobile CDP capture, assembles the payload, POSTs to the server.
bridge.js	Content script (ISOLATED world)	Message relay. The only context that can talk to both the service worker (chrome.runtime) and the page (window.postMessage).
content.js	Injected into MAIN world	The extractor. Full access to the page DOM, getComputedStyle and getBoundingClientRect. Cannot use chrome.* APIs, so it talks only via postMessage.

Why the MAIN world matters. content.js must run in the page's own JavaScript context because that is the only place getComputedStyle() returns the fully-resolved values the page is actually rendering with. An isolated content script sees a different view. But the page cannot use extension APIs - so bridge.js exists purely to shuttle messages across that boundary. This single constraint explains the whole three-script design.

Injection sequence (background.js, ensureEngineLoaded):

13. Check session storage: is the engine already loaded in this tab? If yes, skip.
14. Fetch the engine source from the local server at /engine/content.js.
15. Use chrome.scripting.executeScript to post a SNAPSTAK_INJECT message into the isolated world.
16. bridge.js injects the engine - first as a <script src> pointing at localhost, then falling back to inline injection if CSP blocks the src.
17. content.js loads, posts a ready signal, bridge.js relays CONTENT_READY to the service worker, which marks the tab ready in session storage.

4.3 content.js - where the pillars are born

The largest file in the capture layer (~4,500 lines). It is message-driven: bridge.js sends a command with a requestId, content.js does the work and posts back a matching response. The commands that matter:

Command	What it produces
EXTRACT_PAGE	Full desktop capture: visible elements, hidden elements, hidden interactive components, CSS, JS, Influence, and Phase-1 Objective.
EXTRACT_MOBILE	The same capture under mobile device emulation, for the responsive viewport.
DISCOVER_BREAKPOINTS	Scans every stylesheet for min-width media queries to find the page's real breakpoints, not generic device widths.
START_COMPONENT_SELECT	Interactive mode: the user clicks one component to capture instead of the whole page.

How each pillar is gathered inside content.js:

- **Structure** - the DOM is walked element by element, recording tag, text, class names, ARIA label, a rect from getBoundingClientRect(), and a dictionary of computed CSS properties. Browser-measured text-wrap lines are captured so the SVG can reproduce wrapping exactly. Images and SVG icons are resolved and prefetched (waitForImages, prefetchSVGSprites). The result is a flat element array with parent links.
- **Behaviour** - matched CSS rules and any JavaScript handlers are captured as base64-encoded blobs on the page map. These are the source the client's Behaviour AI later describes in words.
- **Influence** - captureInfluence() parses the user agent into browser and OS name and version, and records screen size, viewport, device pixel ratio, prefers-color-scheme and prefers-reduced-motion.

- **Objective (Phase 1)** - captureObjective() infers a device type from the viewport and offers standard screen presets. The fields the model truly needs (framework, target width, device) are deliberately left null; they are filled in Phase 2 when the user confirms them in the app.

Two-phase capture for responsiveness. A single desktop snapshot cannot describe a responsive layout. So after the desktop capture, background.js attaches the debugger, calls Emulation.setDeviceMetricsOverride to emulate a phone at a real discovered breakpoint, waits for reflow, and runs EXTRACT_MOBILE. The mobile snapshot rides along in the payload's viewportSnapshots array. The client later turns it into a second Structure SVG and a filtered mobile Behaviour file, which is what lets the generated code carry correct @media rules.

4.4 The payload and the hand-off

background.js assembles one JSON object with all four pillars plus both viewports, stamps a componentId of the form `hostname_unixtimestamp`, and POSTs it to `http://localhost:5174/api/snapshot`. That POST is the boundary between the extension and the .NET system; everything after it is C#.

popup.js is the thin toolbar UI: a Deconstruct button that sends EXTRACT_PAGE_RESPONSIVE to the service worker, and a Select button that sends START_SELECT. It does no work itself - it just triggers the background worker.

5. The Server (Host and Relay Layer)

Location: `SnapStak.Wasm.Server/`. An ASP.NET Core Minimal API on :5174 with three distinct roles. Keep them separate in your head.

18. **Static host.** Serves the Blazor WASM client (UseBlazorFrameworkFiles, UseStaticFiles) and the capture engine to the extension at /engine/content.js.
19. **Real-time relay.** Receives snapshots from the extension and pushes them to the running WASM app over Server-Sent Events.
20. **Disk pipeline and integrations.** Runs its own copy of the pillar pipeline that mirrors components to the filesystem, and hosts the design-tool relays (Canva, Framer, Figma) and the mobile pairing / file transfer servers.

5.1 Program.cs - startup and configuration

On startup the server loads `snapstak_settings.json` (if present) for the output root and plugin toggles, configures the storage root and the pipeline, opens CORS to any origin (local-tool convenience), raises the Kestrel timeouts to two minutes for the mobile subnet scan, and starts the mobile pairing and file-transfer servers. It then maps the endpoint set below.

5.2 Endpoint reference

Endpoint	Method	Purpose
/api/snapshot	POST	Receives the capture payload from the extension. Relays to SSE clients, then runs the disk transform on a background task.
/api/events	GET	Server-Sent Events stream. The WASM client subscribes here to receive snapshots in real time.
/health	GET	Liveness check used by the extension's online indicator.
/api/canva/auth, /callback, /send, /status	GET/POST	Canva Connect OAuth and "Send to Canva". Credentials from environment (canva.env).
/api/framer/send, /status	GET/POST	Relay a captured design to Framer.
/api/figma/svg	GET	Serves the master SVG for the Figma importer plugin.
/api/settings	GET/POST	Read and write snapstak_settings.json - output path and plugin toggles.
/discover, /pair	GET/POST	Mobile device discovery and pairing on the local network.
/api/mobile/pairing/*	GET/POST	PIN generation, pairing status, paired-device list, device removal.
/receive, /api/mobile/files/*	GET/POST	Mobile file transfer: receiving captures from a paired phone.

5.3 The snapshot relay (the SSE bridge)

The two endpoints that carry the live capture are `POST /api/snapshot` and `GET /api/events`. The ordering inside /api/snapshot is deliberate:

21. Keep the most recent snapshot in memory.
22. Strip the large viewportSnapshots array from the SSE copy - those mobile snapshots are ~500KB and would cause deserialization failures in the WASM client, which has had them processed server-side already.
23. Push the trimmed snapshot to every connected SSE client immediately - before any transform - so the UI updates with no perceived delay.
24. Run the disk-writing Pipeline.TransformAsync on a background task so the relay is never blocked.

On the client, `/api/events` is consumed with the browser's native EventSource API (see `SnapStakSse.js` and `SnapshotBridgeService`) because HttpClient streaming is unreliable in Blazor WASM. A 20-second keepalive comment holds the connection open.

5.4 The server-side pipeline (SnapStakPipeline.cs)

This ~1,900-line file is the server's mirror of the client pipeline. It deserialises the same DOM payload, builds the same SVG, and writes the pillar files to the configurable output root on disk (default `C:\ConteX Dev\SnapStak.Wasm\output`). It uses SkiaSharp for rasterisation and references the client project directly so it can run the same translator plugins. If you are tracing why a file appears on disk, this is the code; if you are tracing why the in-browser app shows something, look at the client. They share the data models but are otherwise independent.

6. The Client (Blazor WebAssembly App)

Location: `SnapStak.Wasm.Client/`. The application the user works in, compiled to WebAssembly and running entirely in the browser. It owns the four pillars after capture, runs the AI, and produces the code. This is the project to understand most deeply.

6.1 Folder map

Folder	Contents
`Services/`	Engine logic: the pipeline orchestrator, the four pillar agents, AI gateway, licensing, model fetch, mobile services.
`Engine/StructureAgent/`	Pure logic turning a DOM element array into an SVG node tree (StructureService) and serialising it (SvgSerializer).
`Engine/Constructor/`	SvgDeserializer - reads the SVG back into objects for code generation.
`Engine/RulesEngine/Prompts/`	Prompt builders. ConteXCodePrompt.cs is the literal implementation of ConteX Law.
`Engine/Plugins/`	The translator plugin contract and the Figma / Canva / Penpot plugins.
`Models/`	Plain data types by pillar: Dom, Css, Svg, Pillars, Session, Requests.
`Storage/`	IndexedDB-backed pillar storage (IPillarStorage / IndexedDbPillarStorage) and the encryption wrapper.
`Pages/`	Blazor UI: GetStarted, Transform, Settings, Sessions, Transfer.
`Layouts/`	MainLayout with the top navigation.
`wwwroot/js/`	JS interop helpers for IndexedDB, SSE, Web Crypto, the capture bridge, and downloads.
`wwwroot/engine/`	content.js - the capture engine served to the extension.

6.2 Startup: App.razor and the route guard

App.razor runs before any page renders and acts as the gate. On initialise it validates the subscription (skipped in Development, which only checks local state), then decides where to send the user. If the licence is not ready it redirects to `/getstarted` with a status-specific message (past due, cancelled, paused, and so on). If the licence is ready but the user is landing on the root, it checks the OpenRouter credit balance and routes either to the Transform page or to Settings to top up. Only once this check completes does the Router render. A developer debugging "why did it bounce me to Get Started" should start here.

6.3 Routing and the pages

Route	Page	Purpose
/	Transform.razor	The main workspace. Receives the live snapshot, shows the captured component, lets the user pick framework / style / language / unified mode, and runs Convert. Also hosts the Send-to-Framer and Send-to-Canva actions.
/getstarted	GetStarted.razor	Onboarding: enter the OpenRouter API key and activate the subscription (by code, or polled by email after checkout).

Route	Page	Purpose
/settings	Settings.razor	Manage the API key, view subscription status, set the output path, and toggle the design-tool plugins (Penpot, Canva, Framer, Figma).
/sessions	Sessions.razor	Lists captured components from the session manifest.
/transfer	Transfer.razor	Mobile device pairing and file sync.

6.4 Dependency injection - Program.cs

Program.cs registers every service and is the best file to read first, because it shows how the pieces depend on one another. The registrations that carry real meaning:

- **PillarEncryption keyed to environment** - enabled:false in Development, enabled:true in production (section 9.2).
- **Two separate HttpClients** - one for openrouter.ai with a 3,600-second timeout for long generations, one for the local server for plugin and settings sync. Kept distinct so AI and local traffic never share configuration.
- **TranslatorPluginHost via reflection** - plugins are discovered by scanning loaded assemblies for IConteXTranslatorPlugin, because Blazor WASM cannot load .dll files at runtime; every plugin is compiled in.
- **ConteXPipelineService on top** - the single entry point the pages call, depending on all four agents plus licensing, models and storage.

6.5 The pipeline orchestrator (ConteXPipelineService)

The conductor. The UI never calls the agents directly; it calls this service, which exposes the two halves of the flow:

- **TransformAsync** - the deconstruct half. Resolves the user UUID, runs the Structure agent, and registers the component in the session manifest so it appears on the Sessions page. Reports progress through IProgress<PipelineProgress> so the UI bar moves without polling.
- **GenerateAsync** - the convert half. Resolves the API key, fetches the live model list (also the usage-analytics ping), picks the Behaviour and Constructor models by tag, and calls the Constructor. Fails early with a clear message if no key is configured.

Read this file first. Its class comment lays out the four stages, and every other service is something it calls.

6.6 The Transform page in detail

Transform.razor is the workspace and a good study of how the client behaves. On initialise it restores the last snapshot synchronously (so the first render shows data with zero delay), then asynchronously checks readiness, re-runs the transform to guarantee the pillar files are in IndexedDB, connects the SSE bridge to receive new snapshots, and checks Framer and Canva connection status. When the user clicks Convert, `Generate()` calls the orchestrator with the chosen options and a progress callback; `DownloadZip()` hands the resulting bytes to a JS download helper. The INSUFFICIENT_CREDITS error is caught and turned into a friendly top-up message - a small but telling example of the error handling throughout the UI.

7. The Four Pillars in Code

Each pillar maps to specific services and files. This is the heart of the engine and where a new developer should spend the most time.

7.1 Pillar 1 - Structure

Files: `StructureAgentService.cs`, `StructureService.cs`, `SvgSerializer.cs`. Goal: turn the flat DOM element array into one SVG that is the immutable source of truth for geometry, hierarchy, colour and type. Three steps:

25. **Build the tree.** `StructureService.BuildSVGTree` converts the flat element list into a parent-child node tree using parent links, then sorts children by Y then X so document order is deterministic. This determinism is not incidental - it is what makes the eventual output reproducible.
26. **Apply styles.** `ApplyCssProps` merges the captured computed-style dictionary onto each node.
27. **Serialise.** `SvgSerializer.SerializeTreeSVG` walks the tree and translates CSS facts into SVG facts: box-shadow becomes an feDropShadow filter, borders become strokes, border-radius becomes rx (capped for pill shapes), text wrapping becomes tspan lines using the browser-measured wrap data, and fixed/sticky elements lift into a separate overlay layer.

Why SVG, not HTML. SVG forces every element to carry an exact position and size, leaving no ambiguity for the model to resolve later. StructureAgentService also runs every translator plugin against the same tree (producing a Penpot or Canva file alongside the SVG) and processes the mobile snapshot into its own SVG. All output is written to IndexedDB through the storage layer.

7.2 Pillar 2 - Behaviour

File: `BehaviourAgentService.cs`. Goal: describe in words what the captured CSS and JavaScript do, so the model understands behaviour rather than re-deriving it. This is the one pillar that uses AI during deconstruction. It runs two prompts in parallel - one for CSS, one for JS - through the RulesEngineService prompt builders, and writes `_css.md` and `_js.md` description files. It skips empty work: with no JavaScript, no JS description is generated. These markdown descriptions, not the raw CSS, feed the Behaviour section of the final prompt.

7.3 Pillar 3 - Influence

File: `InfluenceObjectiveService.cs` (InfluenceService). Goal: record the rendering environment so the generated code can honour it. Influence is captured in the browser and is storage-only on the .NET side - no AI. It carries browser and OS, viewport and device pixel ratio, and the colour-scheme and reduced-motion preferences. At generation time these appear verbatim in Pillar 3 of the prompt, so a page captured in dark mode is regenerated knowing that.

7.4 Pillar 4 - Objective

File: `InfluenceObjectiveService.cs` (ObjectiveService). Goal: state what the output must become. This is the only pillar driven by the developer rather than the page. `UpdateObjective` is called at convert time with the chosen framework, target width, device, the unified-vs-separate mode instruction, and the captured breakpoint widths. Those breakpoints matter: the generated @media

queries are written against the page's real breakpoints, not generic device ranges. Without a stated Objective there is no definition of a wrong answer - this pillar is the target the other three are held to.

8. Code Generation

Where the four pillars converge into a single AI call and become code. Two files do the work: ConteXCodePrompt.cs assembles the prompt, ConstructorService.cs runs it and packages the result.

8.1 ConteXCodePrompt.cs - ConteX Law as a prompt

The literal implementation of the law. Its class comment quotes it: "Structure + Behaviour + Influence + Objective fed to AI eliminates hallucination. In any domain. Every time." The Build method assembles one prompt with a delimited section per pillar:

- **Pillar 1 (Structure)** - the SVG skeleton with a legend telling the model exactly how to read each attribute (translate = position, data-w/data-h = size, data-gap, data-display, inkscape:label = component type, and more). The instruction is explicit: the SVG is immutable fact; transcribe, do not interpret.
- **Pillar 2 (Behaviour)** - the CSS and JS markdown descriptions, with an explicit fallback line when a description is absent so the model never guesses silently.
- **Pillar 3 (Influence)** - the formatted environment block.
- **Pillar 4 (Objective)** - target framework, device, width and captured breakpoints.

It then appends a framework-specific instruction block (React hooks vs Vue composition API vs Svelte reactivity, plus Tailwind rules when selected) and fourteen numbered output rules. The most consequential is the first: return only a JSON object with exactly `componentCode` and `componentCSS` - no prose, no markdown fences. A constrained output format is itself part of closing the gap.

8.2 ConstructorService.cs - run, parse, package

The assembly line that turns the prompt into a downloadable project. GenerateAsync, read top to bottom, is the clearest summary of the whole convert phase:

28. Validate the request (componentId, uuid, framework).
29. Update the Objective pillar with the user's selections and the captured breakpoints.
30. Load all four pillars from storage: the SVG, Behaviour descriptions, Influence, Objective, the raw CSS, hidden elements and source HTML.
31. If the Behaviour descriptions are missing, run Behaviour AI now to create them - generation is self-healing.
32. Deserialise the SVG (SvgDeserializer) and build a cleaned skeleton: strip defs, swap image fills for readable [image:path] tokens, replace icon SVGs with <img> references, drop filters and clip-paths the model does not need.
33. Build the ConteX Law prompt from all of the above.
34. Call the model once through the AI gateway.
35. Parse the JSON response (with a regex fallback if the model wraps it), sanitise the component's function name to PascalCase, and build the project zip in memory.
36. Return the zip bytes plus stats; the page hands them to the browser as a download.

In-memory zip. Unlike the server pipeline, the client never touches a filesystem - BuildZipInMemory writes the component file, an optional CSS file, icon and image assets and a README into a

ZipArchive over a MemoryStream, and the bytes go to a JS download helper. A clean illustration of how the WASM client keeps everything in the browser.

8.3 The AI gateway and model service

AiGatewayService calls OpenRouter directly from the browser using the user's own API key. The key and the prompt never pass through any SnapStak server - a deliberate privacy and cost property. It retries on transient HTTP codes (429, 5xx) with exponential backoff, surfaces a clear INSUFFICIENT_CREDITS error on 402, and runs at a low temperature (0.1) to favour determinism.

ModelService fetches the active model list from models.snapstak.ai before each generation. This indirection lets models be added, removed or promoted without shipping a new WASM build, and the fetch doubles as a usage-count ping. If the endpoint is unreachable it falls back to a hardcoded default set, so a firewalled or offline user is never blocked. Models are selected by tag - one tagged Behaviour, one tagged Constructor - so roles are decoupled from any specific model ID.

9. Storage, Licensing, and Security

9.1 Pillar storage (IndexedDB)

The client persists every pillar file in the browser's IndexedDB behind the `IPillarStorage` interface (implementation `IndexedDbPillarStorage`). Files are keyed under a per-user, per-component path so a component's SVG, descriptions, Influence, Objective and viewport variants live together. The low-level access is a small JS helper (`SnapStakIdb.js`); values handed to it are already encrypted by the C# layer, so the JS never sees plaintext. The full method surface is in Appendix A.

The session manifest. One non-encrypted `manifest.json` per user tracks each captured component - its id, label, zone, file inventory and per-pillar readiness flags (structure / behaviour / influence / objective). It is deliberately unencrypted because it holds only ids and path references, no pillar content, and must stay readable during assembly without the pillar key. This is what the Sessions page reads.

9.2 Client-side encryption (PillarEncryption + SnapStakCrypto.js)

Pillar data is encrypted at rest in the browser using AES-GCM 256 via the Web Crypto API. The key is derived with PBKDF2 (SHA-256, 100,000 iterations, a random per-device salt) from the user's subscription token, and held in memory only - never written to storage. The stored format is Base64 of IV + ciphertext + auth tag. In Development the scheme is bypassed with a "plain:" prefix so data is readable; crucially, plain-prefixed values are always read as plaintext even in production, so a dev database never crashes a production build.

9.3 Engine encryption (ENCRYPT.md, encrypt.js)

The capture engine itself can be shipped encrypted. In Development the client fetches `content.js` as plaintext; in a Release publish it fetches `content.enc`, decrypts it in memory with an AES-256-GCM key, and sends the decrypted source to the bridge without ever writing it to disk. `content.js` is excluded from the production publish. The `encrypt.js` Node script and `ENCRYPT.md` document the steps: generate a 32-byte hex key, run `node encrypt.js --key <hex>` to produce content.enc, set the key in the client, then publish. This is the one step that requires Node.js.

9.4 Licensing (LicenceService)

LicenceService manages the API key, the user UUID, and subscription validation. On startup it validates the stored subscription against subscriptions.snapstak.ai using a challenge-response handshake (a server nonce and HMAC, plus a client nonce). On success it derives the encryption key from the returned subscription token. Its defining policy is **fail-open**: if the validation server is unreachable, the app keeps working on cached local state rather than locking out a paying user during an outage, relying on payment webhooks to invalidate the cache on the authoritative path. Note this as a deliberate availability-over-strictness trade-off.

10. The Translator Plugin System

CON10X can emit more than framework code. The translator plugin system (`Engine/Plugins`) lets the same Structure tree be exported to design tools. Plugins implement `IConteXTranslatorPlugin`

and are discovered by reflection at startup. The contract is two-phase: Phase 1 (DeclareResources) lets a plugin list the remote URLs it needs so the host can pre-fetch them in parallel; Phase 2 (Translate) runs with those bytes in hand. Each plugin returns bytes and a file extension, and the storage layer writes one extra file per plugin alongside the master SVG (`WriteTranslatorOutput`).

Plugin failures are isolated: a broken plugin logs and is skipped, never blocking the others or the main pipeline. The shipped plugins are Figma, Canva (a PDF consumed by the Canva relay) and Penpot (a binary archive). The same plugins run in both the client and the server because the server references the client project. Plugins can be toggled and given output paths on the Settings page, which syncs the choices to the server.

11. Getting Started as a Developer

11.1 Suggested reading order

Read the code in this order rather than alphabetically:

37. `Program.cs` (client) - the DI wiring; shows how everything connects.
38. `ConteXPipelineService.cs` - the conductor; its comment is the map of the engine.
39. `ConteXCodePrompt.cs` - ConteX Law made concrete; the four pillar sections.
40. `StructureAgentService.cs` + `StructureService.cs` - how the page becomes the SVG.
41. `ConstructorService.cs` - how the pillars become a downloadable project.
42. `content.js` (extension) - how the raw material is captured.
43. `Program.cs` (server) - the endpoints and the relay.

11.2 Mental models to keep

- **The AI is called once, last, with no gaps.** Everything upstream exists to make that one prompt complete. If output is wrong, the fix is almost always a missing or imperfect pillar, not the model.
- **Determinism is engineered.** Sorted tree order, the SVG as immutable truth, low temperature, and a constrained JSON output format all exist to make the same input produce the same output.
- **Two pipelines, one logic.** Client (IndexedDB, in-memory zip) and server (filesystem mirror) implement the same pillar concepts. Always know which process you are in.
- **The browser is the runtime.** The client is WebAssembly; it talks to the page through a postMessage relay and to the AI directly with the user's own key. No pillar data and no key leave the user's machine except the AI call they pay for themselves.

11.3 Common gotchas

- **Missing-reference errors on a clean build** - the Client must build before the Server; see section 2.3.
- **"Bounced to Get Started"** - the App.razor route guard found no valid licence or no API key; see section 6.2.
- **Snapshot never arrives** - the server must be running before the extension is used, and the SSE connection (SnapshotBridgeService) must be open. Check the /health indicator.
- **Extension shows offline** - the server is not on :5174, or the extension was loaded without Developer mode.
- **NuGet** - always via the Visual Studio 2022 Package Manager UI; never hand-edit the .csproj.

Appendix A: The IPillarStorage API

Every pillar read and write goes through this interface (`Storage/IPillarStorage.cs`). Knowing its shape tells you exactly what is persisted and how it is named.

A.1 Path resolution

Method	Returns
ResolveComponentDir(userUuid, componentId)	The storage path (key prefix) for one component's files.
ResolveIconsDir(componentDir)	The icons sub-path for that component.
ResolveSessionRoot(userUuid)	The per-user session root.

A.2 Pillar reads and writes

Pillar	Write	Read
Structure	WriteSvg, WriteViewportSvg	ReadSvg
Behaviour (descriptions)	WriteCssMd, WriteJsMd	ReadBehaviour, BehaviourMdExists
Behaviour (source)	WriteCssJson, WriteJsJson, WriteViewportCssJson	ReadCssJson
Influence	WriteInfluence	ReadInfluence
Objective	WriteObjective	ReadObjective
Hidden content	WriteHiddenElements, WriteHiddenComponents (+ SVG variants)	ReadHiddenElements
Source HTML	WriteSourceHtml	ReadSourceHtml
Icons	WriteIcon	ReadIcon
Viewport snapshot	WriteViewportSnapshot	ListViewportSnapshots
Plugin output	WriteTranslatorOutput	(written to component folder)

A.3 Session manifest and lifecycle

Method	Purpose
RegisterComponentInManifest	Adds a captured component to the user's manifest (id, zone, label).
ReadSessionManifest	Reads the whole manifest for the Sessions page.
UpdateComponentStatus / UpdateSessionStatus	Move a component or the session through Pending - Processing - Complete - Failed.
CleanupSessionFiles / CleanupPillarFiles	Remove a session's or a component's stored files.
WriteSnapshot / ReadSnapshot / ReadSnapshotDirect / ClearSnapshot	Persist the last raw snapshot so the workspace survives a reload.

Appendix B: The Data Model

The `Models/` folder holds plain data types, grouped by concern. These are the shapes that flow from the extension, through the pillars, into the prompt.

B.1 DOM models (Models/Dom)

Type	Role
DomElement	One captured element: tag, text, class, ARIA, rect, computed CSS, image/icon sources, border radius, browser-measured text-wrap lines, children.
DomRect	x, y, width, height, top, left from getBoundingClientRect().
DomSnapshot	The whole capture: the element list, page width/height, and a page map of sections (each carrying base64 CSS/JS/HTML).
PageMapEntry	One page section: tag, segment id, label, vertical position, and the base64 source blobs.
PictureSource	A <picture> source candidate: srcset, sizes, media, type.

B.2 Pillar models (Models/Pillars)

Type	Carries
InfluenceData	Browser and OS name/version, screen and viewport size, device pixel ratio, colour-scheme and reduced-motion preferences.
ObjectiveData	Device type, target screen width and label, all-breakpoints flag, the captured breakpoint widths, target framework, and any additional intent.
BehaviourData	The behaviour pillar payload for CSS and JS.

B.3 Request and response models (Models/Requests)

Type	Role
TransformRequest	The deconstruct payload from the extension - all four pillars plus the viewport snapshots and client type.
GenerateRequest	The convert request - component id, framework, style, language, breakpoints, unified mode, resolved key and model.
ConteXCodePromptParams	The bundle handed to the prompt builder - all four pillars plus rendering options.
WasmGenerateResult / GenerateStats	The convert result - the zip bytes, the component name, and stats (duration, model, prompt size, object count).
ViewportSnapshotRequest	A single responsive viewport capture.

B.4 Session models (Models/Session)

Type	Role
SessionManifest	The per-user project record: target settings, the component list, and processing/assembly status.
SessionComponent	One component in the manifest: identity, zone, file inventory, per-pillar

Type	Role
	readiness, status.
PillarStatus	Four booleans - structure, behaviour, influence, objective - showing which pillar files exist.
SessionStatus (enum)	Pending, Processing, Complete, Failed.